

Chapter 49. Storage, Warehouses & Lakehouses

Row versus columnar measured, inside a Parquet file, ACID properly, a Delta table with atomic corrections, time travel and compaction, partitioning, and what a warehouse bill is really made of.

Draft chapter

Contents

Chapter 49. Storage, Warehouses & Lakehouses

Why this matters

In plain English

49.1 Where data sits

49.2 Row versus columnar, measured

49.3 Inside a Parquet file

49.4 ACID, properly

49.5 Table formats: a folder of files that behaves like a table

49.6 Partitioning and file sizing

49.7 Warehouses: storage and compute, separated

49.8 What it costs

49.9 Choosing a layout

Common mistakes and how to spot them

In the real world: the table nobody could fix

Tools

The project: turn the sensor archive into a table

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 49. Storage, Warehouses & Lakehouses

Part V — Data Engineering, Integration & Scale

Chapter at a glance

You will learn to: say where each kind of data belongs: files, object storage, databases, warehouses, lakes, lakehouses · explain row versus columnar storage, and show the difference in file size and in what a query has to read · read a Parquet file's internals: row groups, column chunks, statistics, compression · explain ACID properly, and why plain files don't have it · turn a folder of Parquet into a real table with a transaction log, then overwrite one machine's data atomically, travel back to an earlier version, change the schema safely, and compact small files · choose partition columns and file sizes on purpose · explain how cloud warehouses separate storage from compute, and what actually drives their bills.

Before you start: Chapter 45 (files and Parquet), Chapter 46 (idempotent writes), Chapter 47 (write-audit-publish), and Chapter 48 (partitions, pruning, and the sensor dataset).

Time needed: 12–16 hours of reading and practice, spread over two to three weeks.

Tools: Python 3.12 with `duckdb`, `pyarrow`, and `deltalake` (`pip install deltalake`). No Spark, no cloud account, no JVM.

Practice data: the Chapter 48 sensor archive. The companion script rebuilds one day of it (2025-12-01, 216,000 readings) as CSV, Parquet, and a row-store database table, so the three can be compared directly.

Why this matters

Chapter 48 ended with a story where the engine wasn't the problem: the storage layout was. That's the normal case. Most "we need a bigger cluster" conversations are really "we stored this badly" conversations, and the fix costs a day rather than a monthly bill.

Storage decisions also outlive everything else. A tool can be swapped in a quarter. A format and layout, once a year of data is in it and forty reports read it, is what you live with. Getting it right means understanding four things that this chapter demonstrates rather than asserts: **how columnar files work**, **what ACID actually guarantees**, **what a table format adds to a folder of files**, and **what you're paying for in a warehouse**.

This is also where Part V's promises land. Chapter 46 needed writes that replace a partition atomically. Chapter 47 needed to publish only after checks pass, and to roll back when something slips through. Both of those are storage features, and this chapter is where you get them.

In plain English

Think about how a warehouse stores stock, which Riverstone's does.

- **Loose boxes on a floor** are files in a folder. Anyone can put one down or take one away. Nothing stops two people leaving half-finished piles.
- **Racking with labelled bays** is partitioning: when someone asks for the October stock, you walk to the October bay instead of searching the floor.
- **A stock register** is a **transaction log**: a written record of what's officially in the warehouse right now, and what changed when. The boxes matter, but the register decides what counts.
- **Never counting a half-finished delivery** is **atomicity**: a delivery is either registered in full or not at all.
- **Two people counting at once and both getting the right answer** is **isolation**.
- **The register surviving a power cut** is **durability**.
- **Being able to say what the stock was last Tuesday** is **time travel**.
- **A thousand tiny parcels instead of twenty pallets** is the small files problem: same goods, far slower to handle.

A **lakehouse** is exactly this: cheap floor space (object storage) plus a stock register (a table format) that makes it behave like a proper warehouse.

49.1 Where data sits

Place	What it is	Good at	Poor at
Files on a disk or share	CSV, Excel, Parquet in folders	Simple, universal, cheap	No transactions, no access control, no schema guarantees
Object storage (S3, GCS, Azure Blob)	Files addressed by key, in the cloud	Very cheap, endlessly scalable, durable	Objects can't be edited in place; listing is slow; no transactions by itself
Operational database (PostgreSQL, MySQL)	Row-oriented, transactional (Chapter 12)	Many small reads and writes; running the business	Large scans; analytics over billions of rows
Data warehouse (Snowflake, BigQuery, Redshift)	Columnar, SQL, managed compute	Analytics at scale; concurrency; governance	Cost if used carelessly; less suited to non-tabular data
Data lake	Files in object storage, usually Parquet	Cheap, any format, any engine	Without discipline, a swamp: no transactions, no schema, no history
Lakehouse	A lake plus a table format (Delta, Iceberg, Hudi)	Warehouse behavior on lake storage; many engines	More moving parts to understand

Riverstone's layout by the end of Part V is typical of a mid-sized company: orders and customers in the ERP's PostgreSQL (the system of record), a warehouse for analytics (raw, staging, mart), and the sensor archive as Parquet in object storage, exposed as a lakehouse table so it can be corrected, versioned, and read by whichever engine suits the job.

Watch out: "the warehouse" means two things. In this book, Riverstone's physical warehouse at Bhiwandi Main stores crates; the data warehouse stores tables. The chapter names the physical one explicitly when it means crates.

49.2 Row versus columnar, measured

The most important storage idea is also the easiest to demonstrate: **how the bytes are arranged decides what a query has to read.**

- **Row-oriented** storage keeps each record's fields together. Reading one whole record is cheap; reading one field of every record means touching everything. Operational databases and CSV are row-oriented.
- **Columnar** storage keeps each column's values together. Reading one column of every record is cheap, and values of the same type sit side by side, so they compress extremely well. Parquet, and every analytics warehouse, is columnar.

Row oriented (CSV, operational database)

machin	plant	line_t	readin	temper	pressu	units_	scrap_	reading 1
machin	plant	line_t	readin	temper	pressu	units_	scrap_	reading 2
machin	plant	line_t	readin	temper	pressu	units_	scrap_	reading 3
machin	plant	line_t	readin	temper	pressu	units_	scrap_	reading 4

A query for average temperature must read every box: all 8 fields of all 216,000 rows.

Columnar (Parquet, every analytics warehouse)

machine_id	216,000 values of the same kind, stored together
plant	216,000 values of the same kind, stored together
line_type	216,000 values of the same kind, stored together
reading_ts	216,000 values of the same kind, stored together
temperature_c	216,000 values of the same kind, stored together
pressure_bar	216,000 values of the same kind, stored together
units_made	216,000 values of the same kind, stored together
scrap_flag	216,000 values of the same kind, stored together

Same 216,000 readings
CSV 14.6 MB
SQLite table 16.3 MB
Parquet 1.3 MB
11x smaller, and one column to read

The same query reads one stripe. The other seven are never touched.

Figure 49.1 – The same 216,000 readings, two arrangements. The query is the same; the work isn't.

The companion script writes one day of sensor readings three ways:

```
import json, os, shutil, sqlite3, time
import duckdb, pyarrow.parquet as pq
from deltalake import DeltaTable, write_deltalake
from setup_ch49 import build

rows = build() # writes storage/day.csv, day.parquet, day.sqlite
con = duckdb.connect()
for f in sorted(os.listdir("storage")):
    print(f"{f:<12} {os.path.getsize(os.path.join('storage', f)) / 1e6:>7.1f} MB")
print(f"{rows:,} readings, one day, 25 machines")
```

```
day.csv          14.6 MB
day.parquet      1.3 MB
day.sqlite       16.3 MB
216,000 readings, one day, 25 machines
```

Reading it. Identical data: **14.6 MB as CSV**, **16.3 MB in a row-store database table**, and **1.3 MB as Parquet**. Parquet is about **11 times smaller than the CSV** and **12 times smaller than the row store**, because each column compresses on its own: the plant name repeats, the timestamps increase steadily, and the machine ids come from a set of 25.

All three give the same answer, of course:

```
CSV      = "read_csv('storage/day.csv')"
PARQUET  = "'storage/day.parquet'"

print("average temperature, from Parquet:",
      con.execute(f"SELECT ROUND(AVG(temperature_c), 2) FROM {PARQUET}").fetchone()[0])
print("average temperature, from CSV      :",
      con.execute(f"SELECT ROUND(AVG(temperature_c), 2) FROM {CSV}").fetchone()[0])

lite = sqlite3.connect("storage/day.sqlite")
print("average temperature, from SQLite :",
      round(lite.execute("SELECT AVG(temperature_c) FROM readings").fetchone()[0], 2))
lite.close()
```

```
average temperature, from Parquet: 196.49
average temperature, from CSV      : 196.49
average temperature, from SQLite  : 196.49
```

What differs is the work underneath. To average one column, the CSV reader must read and parse every byte of all eight fields; the row store must walk every row; Parquet reads one column chunk and skips the rest, which Chapter 48's `ReadSchema` line made visible in Spark's plans.

When row storage is right. Fetching one order and its lines by id, writing a new order, updating a status: row storage wins, which is why Riverstone's ERP is PostgreSQL and not a pile of Parquet. The rule of thumb: **row for running the business, columnar for understanding it.**

49.3 Inside a Parquet file

Parquet isn't only "columnar". Its internal structure is what lets engines skip work.

```

meta = pq.ParquetFile("storage/day.parquet").metadata
print("rows          :", f"{meta.num_rows:,}")
print("row groups    :", meta.num_row_groups)
print("columns       :", meta.num_columns)

group = meta.row_group(0)
print(f"row group 0 : {group.num_rows:,} rows, {group.total_byte_size / 1e6:.2f} MB on disk")
for i in range(meta.num_columns):
    col = group.column(i)
    stats = col.statistics
    print(f"    {col.path_in_schema:<14} {col.compression:<7} "
          f"    {col.total_compressed_size / 1e3:>8.1f} KB "
          f"    min={stats.min if stats else '-'} max={stats.max if stats else '-'}")

```

```

rows          : 216,000
row groups    : 2
columns       : 8
row group 0   : 122,880 rows, 0.80 MB on disk
  machine_id   SNAPPY      0.2 KB min=M-01 max=M-15
  plant        SNAPPY      0.1 KB min=Bhiwandi Main max=Bhiwandi Main
  line_type     SNAPPY      0.1 KB min=Blow max=Injection
  reading_ts    SNAPPY    209.3 KB min=2025-12-01 00:00:00 max=2025-12-01 23:59:50
  temperature_c SNAPPY    219.4 KB min=169.88 max=223.06
  pressure_bar  SNAPPY    193.8 KB min=81.36 max=111.66
  units_made    SNAPPY     62.0 KB min=0 max=14
  scrap_flag    SNAPPY     4.1 KB min=False max=True

```

How to read it.

- A **row group** is a horizontal slice of the file, here about 123,000 rows. Engines read whole row groups, so it's the unit of skipping.
- Inside a row group, each column is stored as a **column chunk**, compressed on its own with Snappy. Look at the sizes: `plant` takes 0.1 KB for 123,000 values because it's one repeated string (**dictionary encoding**), while `temperature_c` takes 219 KB because every value differs.
- Each chunk carries **statistics**: minimum and maximum. That's what makes Chapter 48's pushed filters work: a query for readings above 230 °C can skip this row group entirely, because its maximum is 223.06.
- `scrap_flag` costs 4.1 KB for 123,000 booleans, because booleans pack tightly and mostly repeat.

What this means in practice.

- **Sort data by the column people filter on** before writing. Statistics only help when values are grouped: if every row group contains machines M-01 to M-25, no row group can ever be skipped by machine.
- **Choose row group size deliberately** (128 MB is a common target). Tiny row groups mean lots of metadata; huge ones mean less skipping.
- **Don't compress with something the reader can't split** or doesn't support. Snappy is fast and universal; zstd compresses better at some CPU cost; gzip is slower to read.

49.4 ACID, properly

ACID is four guarantees, and they matter more the moment two processes touch the same data.

Letter	Guarantee	What goes wrong without it
Atomicity	A change happens completely or not at all	A job dies after deleting yesterday's files and before writing today's: the day is gone
Consistency	The data respects its rules after every change	Half a partition written, so totals don't reconcile (Chapter 47's tests catch this after the fact)
Isolation	Concurrent readers and writers don't see each other's half-done work	A dashboard runs while a table is being replaced and shows 40% of the numbers
Durability	Once committed, it survives crashes and restarts	A "successful" write lost on a machine restart

Chapter 12 introduced these for PostgreSQL, where you get them by default. On plain files, **you don't have any of them**. A folder of Parquet files has no notion of a commit: `rm` then `write` is two separate acts, and anyone reading in between sees whatever happens to be on disk.

That's exactly the gap Chapter 46 worked around with careful ordering, and Chapter 47 worked around with write-audit-publish. A **table format** closes it properly.

49.5 Table formats: a folder of files that behaves like a table

Delta Lake, **Apache Iceberg**, and **Apache Hudi** all do the same core thing: they keep a **transaction log** beside the data files that records which files make up the table right now, and what changed in each version. Readers consult the log, not the folder. That single move buys atomicity, isolation, time travel, schema evolution, and safe compaction.

This chapter uses Delta through `deltalake` (the Rust implementation), which needs no Spark and no JVM. The ideas transfer directly to Iceberg and Hudi.

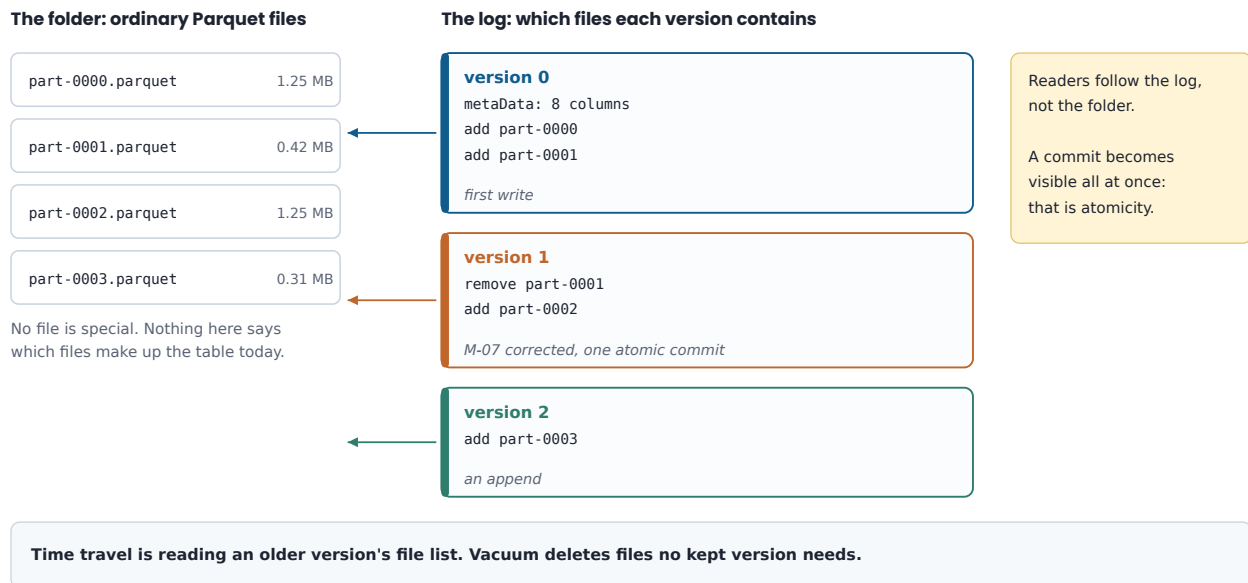


Figure 49.2 – A table format is a folder of ordinary Parquet plus a log that says which files each version contains.

Creating a table

```
shutil.rmtree("delta_readings", ignore_errors=True)
day1 = con.execute("SELECT * FROM 'storage/day.parquet']").arrow().read_all()
write_deltalake("delta_readings", day1, mode="overwrite")

dt = DeltaTable("delta_readings")
con.register("readings_delta", dt.to_pyarrow_dataset())
print("version      :", dt.version())
print("rows         :", f"{con.execute('SELECT COUNT(*) FROM readings_delta').fetchone()[0]:,}")
print("data files    :", len(dt.file_uris()))
print("log entries   :", sorted(f for f in os.listdir("delta_readings/_delta_log") if
                             f.endswith(".json")))
```

```
version      : 0
rows         : 216,000
data files   : 1
log entries  : ['00000000000000000000000000000000.json']
```

Reading it. The table is version 0, holds all 216,000 readings in one data file, and has a `_delta_log` folder with a single JSON entry. The Parquet files are ordinary Parquet: any engine can read them, and the log is what makes them a table.

What's in the log

```
with open("delta_readings/_delta_log/00000000000000000000.json") as f:
    for line in f:
        action = json.loads(line)
        kind = next(iter(action))
        if kind == "add":
            a = action["add"]
            print(f"add      : {a['size'] / 1e6:.2f} MB, {json.loads(a['stats'])['numRecords']:,}
records")
        elif kind == "metaData":
            print("metaData:", len(json.loads(action["metaData"])["schemaString"])["fields"]),
"columns")
        else:
            print(f"{kind:<8}:", {k: v for k, v in action[kind].items() if k in ("operation",
"mode", "minReaderVersion")})
```

```
commitInfo: {'operation': 'WRITE'}
protocol: {'minReaderVersion': 3}
metaData: 8 columns
add      : 1.25 MB, 216,000 records
```

Reading it. Four kinds of action in one commit: `commitInfo` (what operation ran), `protocol` (which reader versions can understand this table), `metaData` (the schema, 8 columns), and one `add` per data file, with its size and row count. A later commit will contain `remove` actions for files that are no longer part of the table. **Nothing is edited in place**; each version is a new log entry describing the change.

An atomic correction

The plant team reports that machine M-07's temperature probe reads 1.5 °C low. That's a correction to one machine's readings, in the middle of a day's data.

```
corrected = con.execute("""
    SELECT * REPLACE (temperature_c + 1.5 AS temperature_c)
    FROM 'storage/day.parquet' WHERE machine_id = 'M-07'""").arrow().read_all()
before = con.execute("SELECT ROUND(AVG(temperature_c), 2) FROM 'storage/day.parquet' WHERE
machine_id = 'M-07'").fetchone()[0]

write_deltalake("delta_readings", corrected, mode="overwrite", predicate="machine_id = 'M-07'")

dt = DeltaTable("delta_readings")
con.register("readings_delta", dt.to_pyarrow_dataset())
print("version now  :", dt.version())
print("rows now      :", f"{con.execute('SELECT COUNT(*) FROM readings_delta').fetchone()[0]:,}"
(the table is whole)")
print("avg temp M-07 before:", before)
print("avg temp M-07 now   :",
      round(con.execute("SELECT AVG(temperature_c) FROM readings_delta WHERE machine_id = 'M-
07'").fetchone()[0], 2))
print("data files    :", len(dt.file_uris()))
```

```
version now  : 1
rows now     : 216,000 (the table is whole)
avg temp M-07 before: 196.0
avg temp M-07 now   : 197.5
data files    : 1
```

Reading it. The write replaced only the rows matching `machine_id = 'M-07'`, and the table went from version 0 to version 1. M-07's average moved from 196.0 °C to 197.5 °C, and the table still holds all 216,000 rows: at no point could a reader see a table missing M-07, because the change became visible only when the commit landed. With plain Parquet files, the same correction means deleting and rewriting files, with a window where the data is wrong or missing.

Watch out: the log is the table. Don't add, delete, or "tidy up" Parquet files inside a table's folder by hand. Readers follow the log, so a file you add is invisible and a file you delete makes the table unreadable. Use the table's own API (append, overwrite, delete, optimize, vacuum).

Time travel

```
con.register("v0", DeltaTable("delta_readings", version=0).to_pyarrow_dataset())
print("version 0, avg temp for M-07:",
      round(con.execute("SELECT AVG(temperature_c) FROM v0 WHERE machine_id = 'M-07'").fetchone()[0], 2))
print("version 1, avg temp for M-07:",
      round(con.execute("SELECT AVG(temperature_c) FROM readings_delta WHERE machine_id = 'M-07'").fetchone()[0], 2))
for entry in DeltaTable("delta_readings").history():
    print("version", entry["version"], "-", entry["operation"], "-",
          entry["operationParameters"].get("mode"))
```

```
version 0, avg temp for M-07: 196.0
version 1, avg temp for M-07: 197.5
version 1 - WRITE - Overwrite
version 0 - WRITE - Overwrite
```

Reading it. Version 0 still shows the original 196.0 °C, and version 1 shows the corrected 197.5 °C. The history lists both commits. Time travel is what makes Chapter 47's promise of a quick rollback real: if a bad load publishes, you restore the previous version instead of rebuilding from source. It's also how you answer "what did this report say last Tuesday?"

Old versions cost storage, so table formats have a `VACUUM` operation that deletes files no longer referenced by any recent version, with a retention window (7 days by default in Delta). Running vacuum with a short retention destroys your ability to time travel, and can break long-running readers.

Schema changes

```
with_shift = con.execute("""
    SELECT *, CASE WHEN EXTRACT(hour FROM reading_ts) < 12 THEN 'A' ELSE 'B' END AS shift
    FROM 'storage/day.parquet' WHERE machine_id = 'M-01'""").arrow().read_all()
write_deltalake("delta_readings", with_shift, mode="append", schema_mode="merge")

dt = DeltaTable("delta_readings")
con.register("readings_delta", dt.to_pyarrow_dataset())
print("columns now:", dt.schema().to_arrow().names)
shift_rows = con.execute("SELECT COUNT(*) FROM readings_delta WHERE shift IS NOT NULL").fetchone()[0]
print("rows with a shift value:", f"{shift_rows:,}")
```

```
columns now: ['machine_id', 'plant', 'line_type', 'reading_ts', 'temperature_c', 'pressure_bar',
'units_made', 'scrap_flag', 'shift']
rows with a shift value: 8,640
```

Reading it. The appended rows carry a new `shift` column. With `schema_mode="merge"`, the table's schema grows: existing rows have `shift` as NULL, the 8,640 new rows have a value. Adding a column is safe. Renaming or removing one, or changing a type, is not: those need a new version of the table's contract with the people reading it (Chapter 47).

Compaction, and the small files problem

Streaming and frequent appends produce many small files. Every reader then pays for opening each one, which on object storage means a network round trip apiece.

```
shutil.rmtree("delta_small", ignore_errors=True)
batch = con.execute("SELECT * FROM 'storage/day.parquet' LIMIT 9000").arrow().read_all()
for i in range(24):
    # 24 small appends, as a streaming job would make
    write_deltalake("delta_small", batch, mode="append" if i else "overwrite")

def file_report(table, label):
    dt = DeltaTable(table)
    sizes = [os.path.getsize(u.replace("file://", "")) for u in dt.file_uris()]
    print(f"{label}: {len(sizes)} files, average {sum(sizes) / len(sizes) / 1e3:.0f} KB, "
          f"total {sum(sizes) / 1e6:.2f} MB")
    return dt

dt = file_report("delta_small", "before")
dt.optimize.compact()
dt = file_report("delta_small", "after ")
con.register("small_delta", DeltaTable("delta_small").to_pyarrow_dataset())
print("rows unchanged:", f"{con.execute('SELECT COUNT(*) FROM small_delta').fetchone()[0]:,}")
print("table version :", DeltaTable("delta_small").version())
```

```
before: 24 files, average 114 KB, total 2.73 MB
after : 1 files, average 529 KB, total 0.53 MB
rows unchanged: 216,000
table version : 24
```

Reading it. Twenty-four appends left **24 files averaging 114 KB**; compaction merged them into **1 file of 529 KB**, and the total shrank from 2.73 MB to 0.53 MB because merged data compresses better. The row count is unchanged, and the compaction is itself a new version, so readers switch over atomically.

Real tables aim for files of **128 MB to 512 MB**. Run compaction as a scheduled maintenance job (Chapter 46), not by hand, and pair it with vacuum to remove the files it replaced.

49.6 Partitioning and file sizing

Chapter 48 showed partition pruning saving 91 folders out of 92. Choosing the partition column is therefore one of the highest-leverage decisions in a platform.

Choose the column that most queries filter on. For events, readings, orders, and logs, that's almost always a **date**. Secondary partitioning (date and plant) helps only if queries regularly filter on both.

Don't over-partition. A partition per machine per hour across a year is 219,000 folders, each with a tiny file. Listing them takes longer than reading the data, and every engine slows down. Rules of thumb:

Guide	Why
Aim for partitions of at least a few hundred MB	Small partitions waste more time in overhead than they save in skipping
Keep total partitions in the thousands, not millions	Listing and metadata dominate above that
Target files of 128–512 MB	Fewer, larger reads; fewer round trips on object storage
Sort within files by a common filter column	Lets row-group statistics skip work (49.3)
Never partition by a high-cardinality id	One folder per customer is the classic disaster

If queries don't filter on your partition column, partitioning buys nothing. That was the Chapter 48 story: an archive partitioned by machine, queried by date.

Iceberg adds **hidden partitioning** (the table records how partitions derive from a column, so a query filtering on a timestamp prunes daily partitions without the writer's layout leaking into every query), and Delta offers **liquid clustering** as an alternative to fixed partitions. Both exist because fixed partition columns chosen early are hard to change later.

49.7 Warehouses: storage and compute, separated

Cloud warehouses (Snowflake, BigQuery, Redshift, Databricks SQL, Microsoft Fabric) share one architectural idea: **storage and compute are separate**. Your tables live in cheap object storage; queries run on compute you rent by the second, sized independently, and several teams can run at once without fighting for the same machine.

That's why a warehouse can be idle at 3 a.m. and cost almost nothing, and why a badly written query at 10 a.m. can cost real money.

Idea	Snowflake	BigQuery	Redshift
Compute unit	Virtual warehouse (sized XS to 6XL)	Slots (on-demand or reserved)	Cluster or serverless workgroup
Data organization	Micro-partitions, automatic	Columnar storage, partitioning and clustering	Sort keys and distribution styles
You tune by	Clustering keys, warehouse size, auto-suspend	Partitioning, clustering, avoiding <code>SELECT *</code>	Sort/dist keys, vacuum, WLM
Typical billing	Per second of compute, per TB stored	Per TB scanned (on-demand) or per slot-hour	Per node-hour or per RPU-second

Two habits matter in all of them.

- **Prune before you compute.** Filter on the partitioning or clustering column, select only needed columns, and avoid `SELECT *` on wide tables. On a per-TB-scanned model this is literally the bill.
- **Let compute sleep.** Auto-suspend idle warehouses, and don't leave a large cluster running for a dashboard that refreshes hourly.

49.8 What it costs

Cloud bills have four parts.

You pay for	Roughly	Habits that cut it
Storage	Cheap: cents per GB-month	Compress (Parquet), expire old versions, vacuum
Compute	The main cost	Prune, size compute to the job, auto-suspend, avoid rerunning the same query
Data scanned	On some services, the whole bill	Partition, cluster, select fewer columns
Data transfer out (egress)	Small but surprising	Keep processing next to the data; don't export whole tables to laptops

A worked estimate for Riverstone

Suppose the sensor archive grows to one year across 40 machines: about **126 million readings a day-equivalent**, stored as Parquet at roughly the compression this chapter measured (1.3 MB per 216,000 readings, so about **6 bytes per reading**). That's $126,000,000 \times 365 \div 216,000 \times 1.3 \text{ MB} \approx 277 \text{ GB}$ per year of raw readings.

- **Storage:** 277 GB at about 0.023perGB – month(typicalobjectstoragelistprice)isabout * *6.40 a month**, or roughly ₹530 at ₹83 to the dollar. Storage is almost never the problem.
- **Query cost on a per-TB-scanned service:** the weekly plant report reads one week: $277 \text{ GB} \div 52 \approx 5.3 \text{ GB}$. At 5perTBscanned, that's about * *0.027 a run**, a few rupees. The same report written to read the whole archive scans 277 GB, about **\$1.39 a run**, more than fifty times as much, for the same answer.
- **The lesson in one line:** you don't optimize a data platform's cost by buying cheaper storage; you optimize it by not reading data you don't need.

Simplification note. Prices, units, and free tiers change constantly and differ by region and provider. Treat these as an illustration of the *shape* of a bill, not as current pricing: check your provider's calculator, and Chapter 52 covers cloud cost management properly.

49.9 Choosing a layout

Situation	Store it as	Why
The business's live records (orders, customers)	Operational database, row-oriented	Many small reads and writes; ACID by default
Analytics tables that people query all day	Warehouse tables, columnar	Managed, concurrent, governed
Large event or sensor archives	Parquet in object storage, as a table format	Cheap, any engine, correctable, versioned
Data you receive from partners	Raw files, kept as they arrived, plus a typed table	Reprocessing without asking again (Chapter 45)
Intermediate results in a pipeline	The warehouse, or Parquet	Rebuildable; don't over-engineer
A one-off analysis	A local Parquet file	No platform needed

For Riverstone, the honest answer is: keep orders in the warehouse (a few hundred thousand rows will never be a problem), and keep sensor readings as a Delta table in object storage, partitioned by date, compacted weekly, with a year's retention for time travel.

Common mistakes and how to spot them

Mistake	Symptom	Fix
CSV as a storage format for analytics	Slow queries, huge files, type guessing	Parquet, with declared types (49.2)
Editing files inside a table's folder	Readers see stale or broken tables	Use the table API; the log is the table (49.5)
Treating a file overwrite as atomic	A failed job leaves a day missing or half-written	A table format, or write-audit-publish (49.4–49.5)
Never compacting	Reads get slower every week; thousands of tiny files	Scheduled compaction plus vacuum (49.5)
Vacuuming with a short retention	Time travel gone; long-running readers fail	Keep a retention window that matches your rollback needs
Partitioning by a high-cardinality column	Hundreds of thousands of folders; listing dominates	Partition by date; sort within files (49.6)
Partitioning by a column nobody filters on	Pruning never happens	Match partitions to the queries (49.6, Ch 48 story)
<code>SELECT *</code> on wide tables	Big bills on per-TB services	Select what you need
Leaving compute running	A bill with nothing to show for it	Auto-suspend; size to the job (49.7)
Optimizing storage price instead of scan volume	Savings of rupees while scans cost hundreds	Reduce what's read (49.8)
Renaming or dropping columns silently	Downstream reports break	Additive changes; contracts for the rest (49.5, Ch 47)

In the real world: the table nobody could fix

Riverstone's sensor archive lived as plain Parquet in object storage, one folder per day, written by a nightly job.

One Tuesday the plant team asked for a correction: machine M-07's probe had been reading low for six weeks, and every report using it was wrong. The data engineer wrote a job to rewrite those 42 days with corrected temperatures.

It ran for 20 minutes and then failed, because the machine it ran on was restarted during a deployment. By that time it had deleted the files for 19 days and written replacements for 11 of them.

The archive was now in three states: 11 days corrected, 8 days **missing entirely**, and 23 days uncorrected. The nightly job had already run, so some reports had been built from the half-repaired archive. Nobody could say which reports had used which version, because plain files have no history.

Recovering took two days: restoring the missing days from object storage's own versioning (which happened to be enabled), rebuilding the reports, and a difficult conversation with the plant manager about which of last week's numbers had been right.

The fix afterwards was the content of this chapter. The archive became a Delta table: corrections are one atomic commit, a failed job leaves the table exactly as it was, every version is recorded with its timestamp, and "what did this report see on Tuesday?" is a query rather than a guess. Compaction and vacuum run weekly, with retention set to a year.

Six months later a similar correction was needed for a pressure sensor. It was one statement, it took four minutes, and the only sign anything had happened was a new row in the table's history.

What made this work.

- **The failure wasn't unusual;** a deployment restarted a machine. The design assumed that would never happen mid-job.
- **Plain files gave no atomicity and no history**, so neither recovery nor investigation was possible.
- **A transaction log turned a two-day incident into a routine commit.**

Tools

- **Python 3.12** with `duckdb`, `pyarrow`, and `deltalake` (`pip install deltalake duckdb pyarrow`). No Spark, no JVM, no cloud account.
 - **The Chapter 49 companion folder** (`companion/ch49/`): `setup_ch49.py`, which rebuilds one day of the Chapter 48 sensor data as CSV, Parquet, and SQLite. Generate the Chapter 48 dataset first.
 - **Versions used for the outputs shown:** Python 3.12.3, deltalake 1.6.3, DuckDB 1.5.5, PyArrow 25.0.1.
 - **Worth knowing about:** Apache Iceberg (PyIceberg), Apache Hudi, and the warehouse products in section 49.7.
-

The project: turn the sensor archive into a table

Goal: convert the Chapter 48 archive into a versioned, correctable table, and prove the properties this chapter claims.

Option A: Riverstone. Use the sensor archive.

Option B: your own data. Any Parquet dataset you're allowed to use, with at least a few million rows.

Steps

1. **Measure the starting point:** rows, file count, total size, and the time to answer one question.
2. **Create a Delta table** from the archive, partitioned by `reading_date`, and confirm the row count matches.
3. **Correct one machine's readings** for one month with an atomic overwrite, then show the row count is unchanged and the average moved.
4. **Break a write on purpose** (raise an exception between two writes) and show the table is still at the previous version and fully readable.
5. **Time travel:** answer the same question at the version before the correction and after it.
6. **Make small files:** append 50 small batches, measure the file count and average size, compact, and measure again, including query time before and after.
7. **Add a column** with `schema_mode="merge"` and show old rows get NULLs.
8. **Write a retention policy:** how long you keep old versions, when vacuum runs, and what would be lost if it ran with a one-hour retention.
9. **Estimate the monthly cost** of your table at ten times its current size, using your provider's published prices, for storage and for one weekly query, partitioned and unpartitioned.

Stretch goals

- Repeat step 6 with an Iceberg table via PyIceberg and compare the workflow.
- Sort each day's data by `machine_id` before writing, and measure whether queries filtering by machine get faster (check row group statistics).
- Add the Chapter 47 tests to run before each commit, so a failing correction never becomes a version.

You've got it when...

- ☐ You can name six places data can sit and say what each is good and bad at.
- ☐ You can explain row versus columnar storage, and quote a real size comparison.
- ☐ You can read Parquet metadata: row groups, column chunks, statistics, compression.
- ☐ You can explain each letter of ACID and what goes wrong without it on plain files.
- ☐ You can create a table format table, read its log, and say what `add`, `remove`, and `metaData` mean.
- ☐ You can correct part of a table atomically, and explain what a reader sees during the change.

- ☐ You can use time travel to answer "what did it say before?", and say what vacuum would destroy.
 - ☐ You can add a column safely and explain why renaming one isn't safe.
 - ☐ You can explain the small files problem and fix it with compaction.
 - ☐ You can choose a partition column and file size, and defend both.
 - ☐ You can explain how warehouses separate storage from compute and what drives the bill.
 - ☐ You can estimate a monthly cost and show why scan volume matters more than storage price.
-

Recap

- **Row storage** suits running the business; **columnar** suits analyzing it. The same day of readings is 14.6 MB as CSV, 16.3 MB in a row store, and **1.3 MB as Parquet**.
 - Parquet's **row groups**, **column chunks**, **dictionary encoding**, and **statistics** are what let engines skip work; sort by the column people filter on so the statistics can help.
 - **ACID** means atomicity, consistency, isolation, durability. Plain files in a folder have none of them.
 - A **table format** (Delta, Iceberg, Hudi) adds a **transaction log** that lists the files making up each version. The log is the table: never edit files inside it by hand.
 - Table formats give **atomic partial overwrites**, **time travel**, **safe schema evolution**, and **compaction**. A correction to one machine moved its average from 196.0 °C to 197.5 °C as a single new version.
 - **Compaction** turned 24 files averaging 114 KB into 1 file of 529 KB with the same rows; target 128–512 MB files.
 - **Partition by what queries filter on** (usually date); don't over-partition; never partition by a high-cardinality id.
 - Warehouses **separate storage from compute**; bills are driven by compute time and data scanned, not by storage price.
 - The cheapest optimization is **not reading data you don't need**: a weekly report that scans a week instead of a year costs about fifty times less.
-

Practice exercises

Warm-up

1. For each, say whether row or columnar storage fits better, and why: (a) fetching one order and its lines by id; (b) the average temperature by machine for a year; (c) inserting a new order; (d) counting scrap readings per plant per month.
2. Using the sizes in section 49.2, how many times smaller is the Parquet file than the CSV, and than the SQLite table? Give both to one decimal place.

3. Name the four ACID properties, and for each give one thing that can go wrong on plain files in a folder.

Core

4. From the row-group output in section 49.3, explain why `plant` takes 0.1 KB while `temperature_c` takes 219.4 KB for the same number of values. What would happen to `machine_id`'s size if machine ids were random 36-character identifiers instead of M-01 to M-25?
5. A query asks for readings above 230 °C. Using the statistics shown in section 49.3, explain what the engine does with row group 0, and what it would have to do if the file had no statistics.
6. In section 49.5's correction, the table stayed at 216,000 rows and moved to version 1. Describe precisely what a reader querying the table halfway through that write would see, and why.
7. After compaction, the 24 small files (2.73 MB total) became one file of 0.53 MB with the same rows. Give two reasons the total shrank.
8. Riverstone keeps 30 days of time travel. Someone runs vacuum with a retention of 1 hour to free space. List three things that break, and say what you'd have done instead.
9. Using the cost estimate in section 49.8, calculate the monthly cost of scanning the full archive (277 GB) once a day for a month at \$5 per TB, and compare it with scanning one week's data daily. State the ratio.

Stretch

10. Design the storage for Riverstone's sensor archive at 40 machines and five years: partition columns, sort order within files, target file size, compaction and vacuum schedule, and retention. Justify each choice, and say what you'd change if most queries were per-machine rather than per-day.
11. The plant team wants to correct six weeks of readings for one machine, as in the real-world story. Write the sequence of steps you'd follow with a table format, including what you'd test before committing (Chapter 47) and how you'd let people know.
12. Compare Delta and Iceberg for a company that uses both Spark and a cloud warehouse, on: engine support, hidden partitioning, schema evolution, and who maintains the tables. Say which you'd choose for Riverstone and why the answer might differ for a larger company.

Think about it (no code needed)

13. "The log is the table." Explain what that sentence means to a colleague who has been tidying up Parquet files in a table folder to save space, and what they should do instead.
 14. Storage is cheap and compute is expensive, yet most teams spend their optimization effort on compression settings. Why does that happen, and what would you measure first instead?
-

Key terms

row-oriented storage · columnar storage · Parquet · row group · column chunk · dictionary encoding · statistics (min/max) · compression (Snappy, zstd) · ACID (atomicity, consistency, isolation, durability) · transaction log · commit · version · table format · Delta Lake · Apache Iceberg · Apache Hudi · add and remove actions · time travel · vacuum · retention · schema evolution · schema merge · compaction (OPTIMIZE) · small files problem · partitioning · over-partitioning · hidden partitioning · liquid clustering · object storage · data lake · lakehouse · separation of storage and compute · data scanned · egress

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 50, Streaming & Real-Time**, writes continuously into tables like these, which is where compaction and small files stop being theoretical.
 - **Chapter 51, Data Activation**, reads from the published tables to push data into business systems.
 - **Chapter 52, The Cloud, Containers & Infrastructure as Code**, covers object storage, permissions, and cost management in practice.
 - **Chapter 47** gains its rollback: a failed publish can now be reversed by restoring a version.
 - **Chapter 63, Designing Automation & Integration Architecture**, and **Chapter 62** (the economics of data platforms) build on section 49.8.
 - **Part VIII**: storage and file format questions appear in the data engineering interview chapters, and layout design cases in Chapter 77.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) **Row**: all the fields of one record sit together, so it's one read. (b) **Columnar**: one column across many rows, and it compresses well. (c) **Row**: inserting a whole record touches one place; columnar formats are written in bulk, not row by row. (d) **Columnar**: two columns (plant, scrap flag) over many rows, with date pruning.

2. $14.6 \div 1.3 = 11.2$ times smaller than the CSV. $16.3 \div 1.3 = 12.5$ times smaller than the SQLite table. (Exact ratios depend on rounding of the printed sizes; anything close is right.)

3. **Atomicity**: a job dies between deleting old files and writing new ones, leaving the day missing.

Consistency: half a partition is written, so totals don't reconcile with the source. **Isolation**: a dashboard queries the folder while files are being replaced and averages a mixture of old and new.

Durability: a write reported as finished is lost when the machine restarts before the data is flushed.

4. `plant` has only one distinct value in that row group ("Bhiwandi Main"), so **dictionary encoding** stores the value once and then a tiny code per row, which then compresses further. `temperature_c` has around 120,000 distinct floating-point values with no repetition, so there's nothing to fold away. If machine ids were random 36-character identifiers, `machine_id` would grow dramatically: no small dictionary, no repetition, and 36 bytes of high-entropy text per row, so it could easily become the largest column in the file.

5. Row group 0's statistics say `temperature_c` has a maximum of 223.06, which is below 230, so **no row in that row group can match**: the engine skips the whole row group without reading its column chunks. Without statistics, the engine would have to read and decompress the temperature column for every row group and test each value, which is the difference between reading kilobytes of metadata and megabytes of data.

6. A reader sees **version 0, complete and consistent**, until the commit lands, and **version 1, complete and consistent**, afterwards. It never sees a mixture. That's because a reader resolves the table by reading the log: until the new log entry is written, the old entry defines the table, and the new data files sitting in the folder are simply not part of it yet. The commit is a single atomic act of adding one log entry.

7. (1) **Better compression**: 24 separate files each carry their own dictionaries and encode short runs; merged into one file, the same values compress far better, especially repeated strings such as `plant` and `machine_id`. (2) **Less per-file overhead**: each Parquet file repeats schema, footers, and row-group metadata, which is a large share of a 114 KB file. A third reason is that the compacted file can use larger, more efficient row groups.

8. Breaks: (1) **Time travel beyond one hour is gone**, so a rollback after a bad publish is impossible; (2) **long-running readers fail**, because files they're still reading are deleted underneath them; (3) **audits and "what did the report say last week?" questions can't be answered**, and any consumer pinned to an older version breaks. Better: keep retention at least as long as your rollback need (30 days here), run vacuum on a schedule with that retention, and if space is genuinely the issue, compact first and check what the old versions actually cost, which is usually small compared with the risk.

9. Full archive daily: $277 \text{ GB} = 0.277 \text{ TB} \times 5 = \text{*\$1.385 a run*}$, $\times 30 = \text{\$41.55 a month}$. One week's data daily: $5.3 \text{ GB} = 0.0053 \text{ TB} \times 5 \approx \text{*\$0.0265 a run*}$, $\times 30 \approx \text{\$0.80 a month}$. The ratio is about **52 times**, which is exactly the ratio of data scanned ($277 \div 5.3$). The point: the query's cost is set by how much it reads, and partitioning is what decides that.

10. A defensible design: **partition by `reading_date`**, because reports are time-based; **sort by `machine_id`, then `reading_ts`** within each day's files so row-group statistics can skip machines; **target 256 MB files**, which at 40 machines $\times$ 8,640 readings a day is a handful of files per day; **compact weekly** and **vacuum monthly with a retention of 30 days**; **retain** raw data for five years in the table, with cold storage for anything older than two years if costs justify it. If most queries were per-machine rather than per-day, the cleanest answer is *still* to partition by date but sort primarily by machine (so statistics prune machines), or to use Iceberg's hidden partitioning or Delta's liquid clustering to cluster by both; partitioning by machine directly would create 40 folders a day, or 73,000 over five years, with small files in each.

11. A sensible sequence: (1) **Reproduce the problem**: confirm the probe offset with the plant team and agree the exact machine, date range, and correction. (2) **Build the corrected data into a staging table**, not the live one. (3) **Test it** (Chapter 47): row counts match the original range, the correction moved values by the expected amount and no other machine changed, and reconciliation against the source still holds. (4) **Commit one atomic overwrite** restricted to that machine and date range, and note the version number. (5) **Verify** by querying both versions (time travel) and comparing. (6) **Rebuild the affected reports** for the six weeks (Chapter 46's backfill, without re-sending the daily emails), and (7) **tell the people who used the old numbers** which reports changed, by how much, and that a corrected version is available: a labeled correction, exactly as in Chapter 46's story.

12. **Engine support**: both are widely supported; Iceberg has broader native support across warehouses (BigQuery, Snowflake, Trino, Athena) while Delta is strongest in the Spark and Databricks world, with `delta-rs` and Unity Catalog broadening it. **Hidden partitioning**: Iceberg has it (queries filter on a column; the table knows how partitions derive from it), Delta answers with liquid clustering. **Schema evolution**: both support adding, and both handle renames through column ids in their newer protocol versions; Iceberg's column-id model has been there longer. **Maintenance**: both need compaction and expiry; on Databricks, Delta's maintenance is largely managed, whereas Iceberg maintenance depends on the engine and catalog you run. For Riverstone: **Delta**, because the team already uses Spark and Python, `delta-lake` needs no extra services, and the tables are read mostly by DuckDB and Spark. For a larger company with several warehouses and query engines, **Iceberg** is often the safer bet precisely because no single vendor's engine is the centre of gravity.

13. It means the folder of Parquet files isn't the table; the log is, and the log lists exactly which files belong to which version. Deleting a file the log still references doesn't save space in any useful sense: it makes the table unreadable, because a reader will look for a file that no longer exists. Adding a file by hand does nothing either, since no reader will see it. If space is the problem, the right tools are **compaction** (merge small files into large ones, as a new version) and **vacuum** (delete files no version within the retention window needs). Both are operations the table supports and records, so readers never break.

14. Compression settings are visible, quick to change, and feel like engineering; scan volume is invisible unless someone measures it, and fixing it means changing layouts and rewriting queries, which touches other people's work. There's also an anchoring effect: storage bills come with a clear GB number, while compute bills are diffuse. What to measure first: **bytes scanned per query** (or slot/compute seconds), broken down by the top ten most frequent and most expensive queries, and how much of what they scan they actually use after filtering. That measurement almost always points at a missing filter, a wrong partition column, or `SELECT *`, each of which is worth more than any compression codec choice.